pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: A Trade Blotter

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The problem: one page on September's trading

One month of the desk's trades: 2026-09-01 to 2026-09-30

374
trades in the month

12
columns in the file

21
trade dates

The first three trades, seven of the 12 columns

trade_id	trade_date	ticker	side	quantity	price	principal
T0001	2026-09-01	KRIX	BUY	500,000	103.053	515,265.00
T0002	2026-09-01	PLWK	SELL	245,000	95.434	233,813.30
T0003	2026-09-01	TOVR	BUY	25,000	97.875	24,468.75

```
blotter = pd.read_csv(data_folder + 'trade_blotter.csv',
                      parse_dates=['trade_date', 'settle_date'])
print(blotter.shape)   # (374, 12)
```

The desk head asks

1. How many trades, bonds, and dollars?
2. How large is a typical trade?
3. A net buyer or a net seller?
4. With which counterparties?
5. What share of each sector traded?
6. On which days and at which hours?

The page is a description. It says what was traded. It does not rate a bond, a counterparty, or a desk.

Everything in the file is invented: the trades, the bonds, the issuers, the dealer names, and the desk names. No name refers to a real firm or person.

Units: quantity is dollars of face. Price is per 100 of face. Principal is quantity times price divided by 100, in dollars.

Every trade is in one of the 200 bonds of holdings.csv, the portfolio used since week 2. Accrued interest is not in the file.

The routine, on a new table

Step 1, section 15.1.1

First look

Shape, first rows, types, and the dates covered

As in notebook 13

```
blotter.shape
```

Step 2, rest of section 15.1

Sanity checks

Ids, blanks, repeats, and whether principal ties out

As in notebook 13

```
blotter.isna().sum().sum()
```

Step 3, section 15.2

One column

Trade size, then trades by hour and by weekday

As in notebook 13

```
blotter['quantity'].describe()
```

Step 4, section 15.3

Join

Give each trade its sector and grade, by cusip

As in notebook 10

```
pd.merge(..., how='left')
```

Step 5, section 15.4

Two columns

Side, sector, counterparty, and grade

As in notebook 13

```
pd.pivot_table(trades, ...)
```

Step 6, section 15.5

Against the book

Principal traded against market value held

New today: turnover

```
traded / held * 100
```

Step 7, section 15.6

Through the month

Net by desk, and the running net day by day

New today: a running total

```
daily['net'].cumsum()
```

Step 8, the Summary

Report

One page for the desk head: the checks, then six facts

What the desk head reads

Six facts, each a number

The routine is the one from weeks 2 and 3. The table is new: you predict more, and you are told less.

The sanity checks all pass

Check	pandas	Excel	Result
Ids unique	`blotter['trade_id'].is_unique`	`=SUMPRODUCT(--(COUNTIF(A2:A375, A2:A375) > 1))`	0 appear twice
Empty cells	`blotter.isna().sum().sum()`	`=COUNTBLANK(A2:L375)`	0
Buys	`blotter['side'].value_counts()`	`=COUNTIF(G2:G375, "BUY")`	197
Principal ties out	`blotter['quantity'] * blotter['price'] / 100`	`=ROUND(H2 * I2 / 100 - J2, 2)`	0 in every row
Multiple of 5,000	`blotter['quantity'] % 5000`	`=MOD(H2, 5000)`	0 in every row
Bond in the holdings	`blotter['cusip'].isin(holdings['cusip'])`	`=COUNTIF(holdings!$B$2:$B$201, E2)`	1 in every row

Sheet blotter: trade_id in column A through trader in column L, rows 2 to 375. No row is repeated. COUNTIF with SELL returns 177.

374 trades on 21 trade dates, in 128 of the 200 bonds of the portfolio. 72 bonds did not trade.

Gap, calendar days	Trades	Which trades
1	319	settle on the next calendar day
3	41	made on a Friday, settle on Monday
4	14	made on Friday 2026-09-04, settle 2026-09-08

```
gap_days = (blotter['settle_date'] - blotter['trade_date']).dt.days
print(gap_days.value_counts())   # three values, not one
```

A clean result is a result

List the checks that passed, and write each one as an assert, so that it runs again on the next file.

```
assert missing == 0, f'{missing:,} cells are missing'
```

The same in Excel, with 2026-09-07 typed into T1

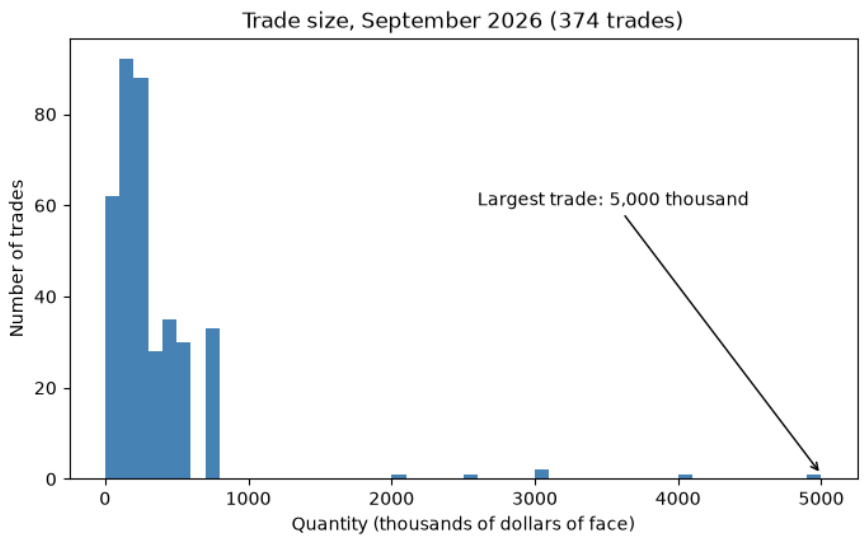
`=WORKDAY(B2, 1, $T$1)` returns the settle date

`=WORKDAY(B2, 1, $T$1) = D2` TRUE in every row

One business day is not one day: no trade on 2026-09-07.

Trade size and the six large trades

Median 200,000 dollars of face, mean 312,968



The same in Excel, with quantity in column H

`=MEDIAN(H2:H375)`	returns 200,000
`=QUARTILE.INC(H2:H375, 3)`	returns 400,000
`=COUNTIF(H2:H375, ">850000")`	returns 6

Chart: Insert, Insert Statistic Chart, Histogram, on column H.

The six trades above the limit of 850,000, largest first

trade_id	trade_date	ticker	side	quantity	price
T0160	2026-09-15	CLAE	SELL	5,000,000	106.702
T0051	2026-09-03	RULL	SELL	4,000,000	103.059
T0367	2026-09-30	VRAN	SELL	3,000,000	92.203
T0237	2026-09-21	PLER	BUY	3,000,000	98.307
T0151	2026-09-14	DRAH	BUY	2,500,000	94.101
T0138	2026-09-11	THAN	SELL	2,000,000	94.181

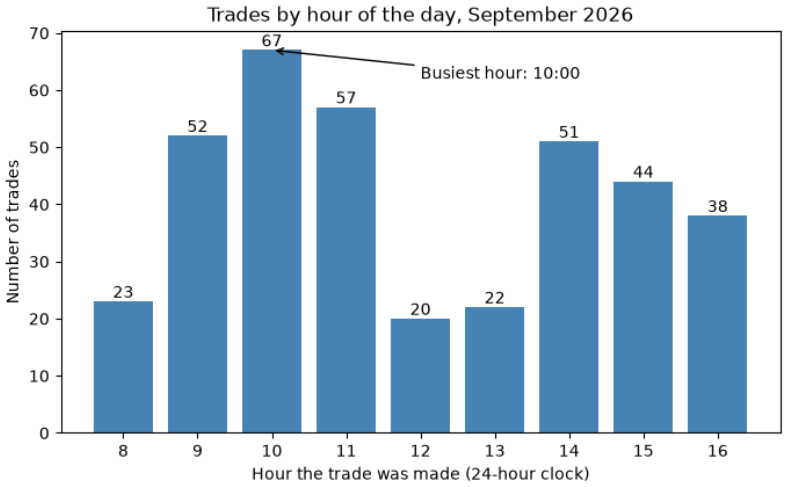
Each has a price near 100 and a principal that ties out. These are large trades: real, and kept. An outlier is not an error.

1.6 percent of the trades, 16.5 percent of the principal traded

```
q1 = blotter['quantity'].quantile(0.25)   # 100,000
q3 = blotter['quantity'].quantile(0.75)   # 400,000
iqr = q3 - q1                             # 300,000
upper_limit = q3 + 1.5 * iqr              # 850,000
```

368 of the 374 trades are of 1,000,000 dollars of face or less.

Trades by hour and by weekday




```
as_time = pd.to_datetime(blotter['trade_time'], format='%H:%M')
blotter['hour'] = as_time.dt.hour
trades_by_hour = blotter['hour'].value_counts().sort_index()
```

By day: most on 2026-09-29, 29 trades. Fewest on 2026-09-28, 7.

A time read as text has no hour

blotter['trade_time'].dt.hour raises an AttributeError: Can only use .dt accessor with datetimelike values. pd.to_datetime(), told the layout with format=, turns the text into a time.

Hours 9 to 11 hold 176 of the 374 trades. Hour 10 has the most, 67. The quietest hours are 12, with 20, and 13, with 22.

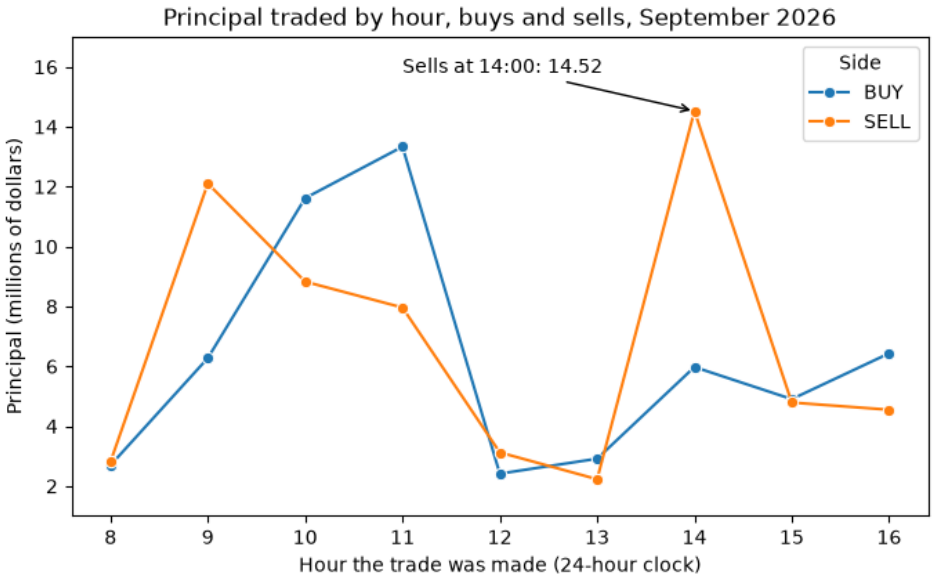
Weekday	Trades	Dates	Trades per day
Monday	27	3	9.00
Tuesday	112	5	22.40
Wednesday	100	5	20.00
Thursday	80	4	20.00
Friday	55	4	13.75

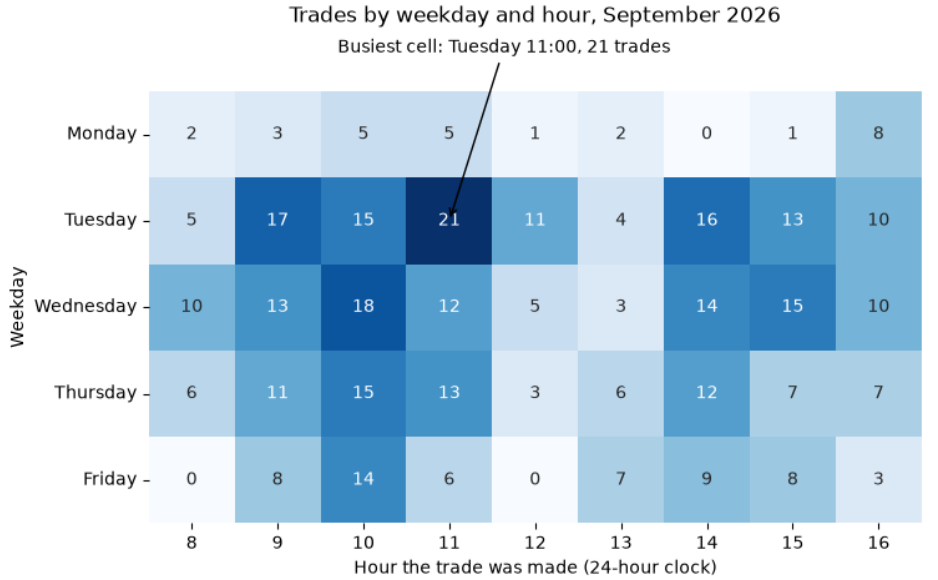
Wednesday, 100 on 5 dates. Thursday, 80 on 4. Both 20.0 a day.

One month, with three to five dates each: not a rule about weekdays.

In Excel `=HOUR(C2)` returns 8 `=COUNTIF(M2:M375, 10)` returns 67 `=VALUE(LEFT(C2, 2))` returns 8 when column C holds text `=TEXT(B2, "dddd")` returns Tuesday `=COUNTIF(N2:N375, "Wednesday")` returns 100. M holds the hour and N the weekday name.

Hour, weekday, and side in two pictures






```
sns.lineplot(x=blotter['hour'], y=principal_mm,
             hue=blotter['side'], estimator='sum')
```

Sells peak at 14:00 with 14.52 million dollars, though the hours 09:00 and 14:00 have 52 and 51 trades. The darkest cell of the heatmap is Tuesday from 11:00, with 21 trades. The hour and weekday charts are the column and row totals of the table.

Joining the blotter to the holdings

The blotter on the left

```
trades = pd.merge(blotter, bond_details, on='cusip',
                  how='left', validate='many_to_one')
```

374 rows before the join, and 374 after

validate= stops the merge if a cusip is in the holdings twice.

Three asserts: the row count, a sector, and a grade.

One row is a trade.

The holdings on the left

```
other_way = pd.merge(holdings, blotter, on='cusip', how='left')
```

446 rows: 374 trades and 72 bonds with no trade

128 bonds traded: one row for each of their 374 trades.

72 bonds did not trade: one row each, with missing values.

One row is a trade, or a bond with no trade.

The first three rows of trades: seven of its 20 columns. The last four were looked up.

trade_id	ticker	side	issuer	sector	rating	grade
T0001	KRIX	BUY	Krilellix Networks	Telecom	BBB-	Investment Grade
T0002	PLWK	SELL	Pellwick Paper	Materials	A-	Investment Grade
T0003	TOVR	BUY	Tovovar Water	Utilities	BBB-	Investment Grade

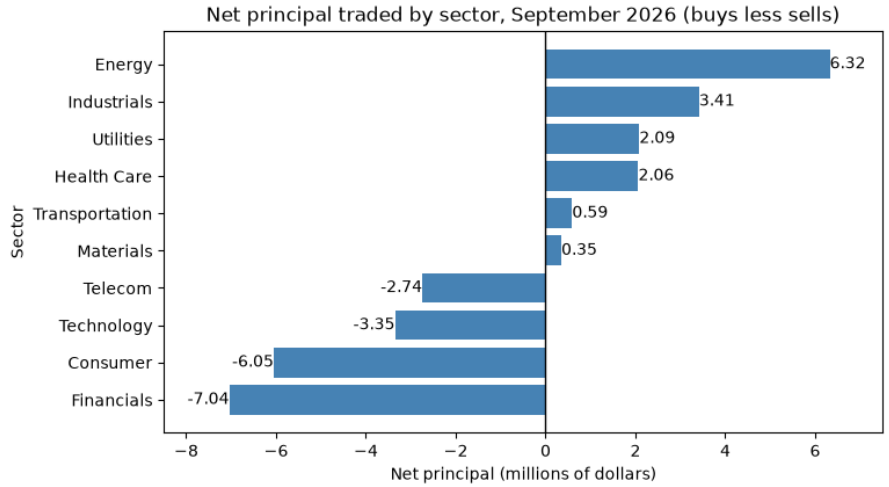
Which table goes on the left decides what one row means.

`=XLOOKUP(E2, holdings!$B$2:$B$201, holdings!$E$2:$E$201)` the sector, in column O. On a repeated key, it returns the first match.

`=SUMPRODUCT(--ISNA(O2:O375))` returns 0. A cusip not on the holdings sheet would show #N/A

Two columns at a time

Net by sector, millions: -7.04 in Financials to 6.32 in Energy




```
by_sector = pd.pivot_table(trades, values='principal',
    index='sector', columns='side', aggfunc='sum')
by_sector['net'] = by_sector['BUY'] - by_sector['SELL']
```

`=SUMIFS(J2:J375, G2:G375, "BUY", O2:O375, "Energy")` 9.29 million

`=MEDIAN(IF(P2:P375 = "High Yield", H2:H375))` 150,000

Pivot table: sector in Rows, side in Columns, Sum of principal

Buys against sells: a count is not a dollar amount

Side	Trades	Dollars, millions
BUY	197	56.56
SELL	177	60.92
Net: buys less sells	20	-4.36
Total: buys plus sells	374	117.47

197 buys, 177 sells, and more dollars sold than bought.

Seven counterparties: first by count, first by dollars

Counterparty	Trades	Principal	Per trade
Kestermoor Securities	86	15,949,252	185,456
Talvesham Partners	58	36,329,404	626,369

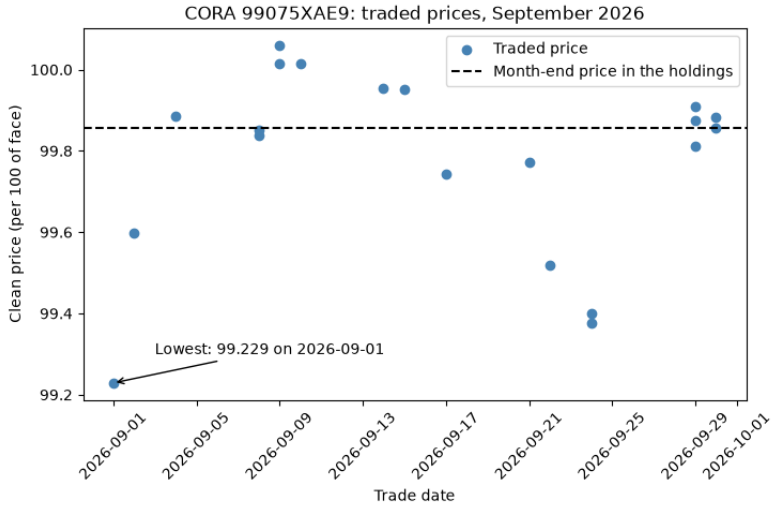
`=SUMIF(K2:K375, "Talvesham Partners", J2:J375)` returns 36,329,404

Size by grade, thousands	Trades	Median	Mean	Largest
Investment Grade	234	250.0	395.9	5,000.0
High Yield	140	150.0	174.4	760.0

Size is dollars of face. All six large trades are Investment Grade.

The blotter against the book

The most traded bond: CORA 99075XAE9, 20 trades in the month



128 bonds traded: 57 once, 25 five times or more, one 20 times.

Simple average 99.777. Volume-weighted 99.742. Month end 99.857.

```
weighted_average = ((top_bond['price'] * top_bond['quantity']).sum()
                    / top_bond['quantity'].sum())   # 99.742
```

`=AVERAGEIF(E2:E375, "99075XAE9", I2:I375)` returns 99.777

Turnover: principal traded as a percent of market value held

sector	traded_mm	held_mm	turnover_pct
Consumer	20.13	56.63	35.55
Energy	12.26	38.83	31.56
Financials	16.13	68.22	23.65
Materials	12.18	71.70	16.99
Transportation	7.24	48.82	14.84
Whole portfolio	117.47	490.82	23.9

Five of the ten sectors, in millions of dollars and percent.

Before adding up a column, ask what one row is

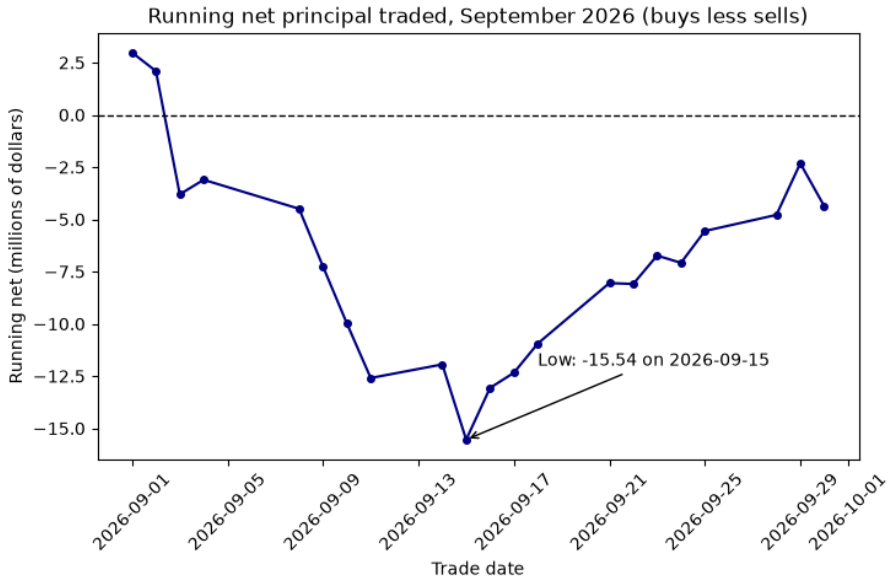
trades['market_value'].sum() gives 1,022.78 million. The portfolio holds 490.82 million. In trades one row is a trade, so a bond that traded 20 times is counted 20 times.

```
traded = trades.groupby('sector')['principal'].sum()
held = holdings.groupby('sector')['market_value'].sum()
```

`=SUMIF(O2:O375, "Consumer", J2:J375)` traded, on the blotter sheet

`=SUMIF(holdings!$E$2:$E$201, "Consumer", holdings!$S$2:$S$201)` held

Through the month




```
daily['net'] = daily['BUY'] - daily['SELL']
daily['running_net'] = daily['net'].cumsum()
```

`=IF(G2 = "BUY", J2, -J2)`	in Q2: the signed principal
`=SUM($Q$2:Q2)`	in R2, filled down: the running total

Net by desk, in millions of dollars

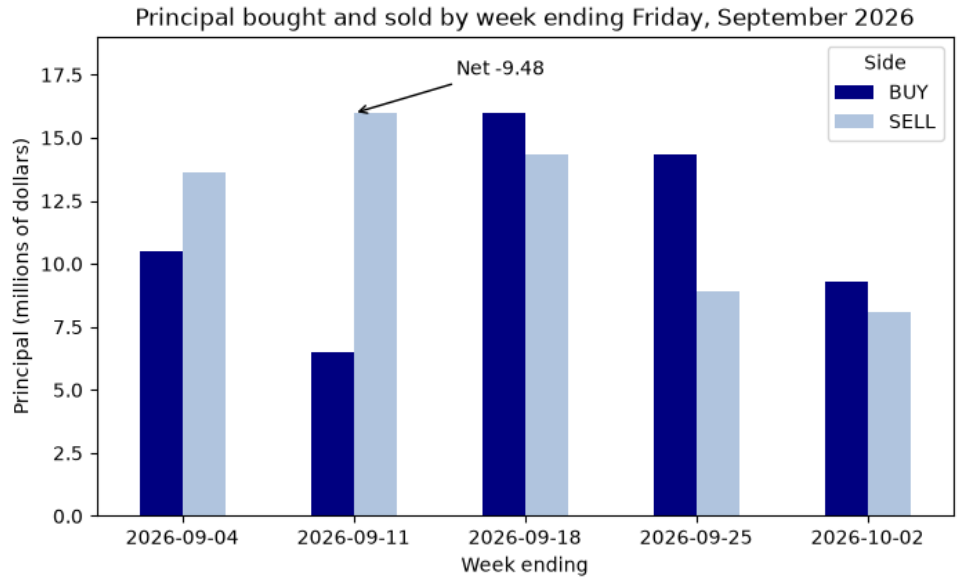
Desk	BUY	SELL	Net
Desk 1	9.75	14.71	-4.96
Desk 2	22.86	12.77	10.09
Desk 3	17.04	20.44	-3.40
Desk 4	6.91	13.01	-6.10
The whole blotter	56.56	60.92	-4.36

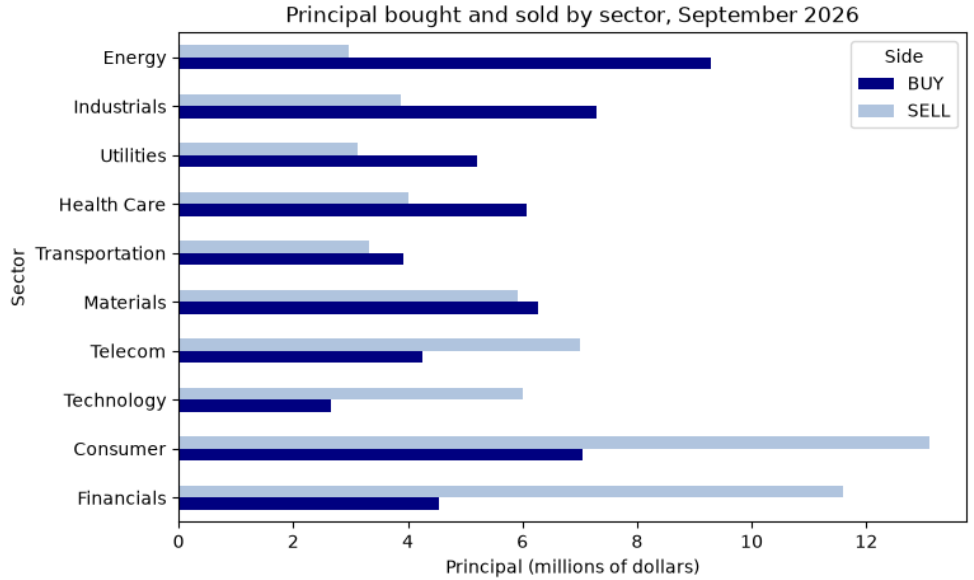
The running net, in millions of dollars

Running net	Millions	Date
Highest	2.98	2026-09-01
Lowest	-15.54	2026-09-15
Month end	-4.36	2026-09-30
Largest net selling in a day	-5.90	2026-09-03

A running total ends at the total. One number for a month hides the path to it.

Week by week, and a chart from a table






```
weekly = daily[['BUY', 'SELL', 'net']]
         .resample('W-FRI').sum()
by_sector[['BUY', 'SELL']].plot.barh(ax=ax)
```

The week ending 11 September had the most net selling, -9.48 million dollars. The five weekly nets add up to -4.36 million, the net of the month. pandas draws a chart straight from a table: one group of bars for each row and one bar for each column.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Run the same routine on a table you have not seen
- Report a clean file by listing the checks that passed, each one an assert
- Flag outliers with the IQR rule, read the rows, and keep the real ones
- Join trades to holdings, and say what one row means
- Report a net and a total, a count and a dollar amount, and a running total

Week 4, Day 1 Case Study: Consumer Credit. A full analysis of a public credit card default dataset